

Module 5

Q. What is Fault Tolerance? Describe Different Types of Failure Models.

=>

Fault Tolerance

1. Fault tolerance is the ability of a distributed system to continue working properly even when some components fail.
2. Fault tolerance helps the system provide services without complete shutdown.
3. Failures may occur due to hardware faults, software bugs, network problems, or power failures.
4. Fault tolerance is important in distributed systems because many machines work together.
5. If one machine fails, the remaining machines should continue functioning.
6. Techniques such as redundancy, replication, backup, and recovery are used to achieve fault tolerance.
7. An example is cloud storage systems where data is stored on multiple servers to avoid data loss.

Failure Models in Distributed Systems

1. A failure model describes the different ways in which a component of a distributed system can fail.
2. Failure models help designers understand faults and build recovery mechanisms.

1. Crash Failure Model

1. In crash failure, a process or machine stops working completely.
2. After crashing, the process does not send or receive any messages.
3. The failed process remains inactive until it is restarted.
4. Crash failure is one of the simplest failure models.
5. An example is a server shutting down due to power failure.

2. Omission Failure Model

1. Omission failure occurs when a process fails to send or receive messages.
2. The process may still be running, but some messages are lost.
3. Omission failures can happen due to network errors or buffer overflow.
4. There are two types of omission failures.

a. Send Omission Failure

1. Send omission failure occurs when a process fails to send a message.
2. The sender creates the message but it never reaches the network.
3. An example is a server failing to send a response to a client.

b. Receive Omission Failure

1. Receive omission failure occurs when a process fails to receive an incoming message.
2. The message is sent but lost before reaching the receiver.
3. An example is a packet being dropped due to network congestion.

3. Timing Failure Model

1. Timing failure occurs when a process responds too early or too late.
2. The process gives the correct result, but not within the required time.
3. Timing failures are important in real-time systems.
4. An example is an online payment confirmation arriving after the allowed time limit.

4. Response Failure Model

1. Response failure occurs when a process gives an incorrect response.
2. The process may reply with wrong data or perform the wrong operation.
3. There are two types of response failures.

a. Value Failure

1. Value failure occurs when the process returns an incorrect value.
2. The operation is performed, but the result is wrong.
3. An example is a banking system showing the wrong account balance.

b. State Transition Failure

1. State transition failure occurs when the process performs the wrong sequence of actions.
2. The process moves to an incorrect state.
3. An example is an ATM dispensing cash without checking the account balance.

5. Byzantine Failure Model

1. Byzantine failure is the most severe type of failure.
2. In Byzantine failure, a process behaves in an unpredictable or malicious way.
3. The failed process may send different incorrect messages to different nodes.
4. Byzantine failures are difficult to detect and handle.
5. An example is a hacked server sending false information to users.

6. Network Failure Model

1. Network failure occurs when communication links between machines fail.
2. Messages may be delayed, lost, or delivered in the wrong order.
3. Network failures can disconnect machines from the system.
4. An example is an internet outage between two data centers.

Q. Different Data-Centric Consistency Models

=>

1. Data-centric consistency models define the rules for maintaining consistency of shared data in distributed systems.
2. These models describe how updates made by one process become visible to other processes.
3. Consistency models are important in distributed databases, cloud systems, and replicated storage systems.
4. Stronger consistency provides more accuracy but may reduce performance.
5. Weaker consistency improves performance but may allow temporary inconsistency.

1. Strict Consistency

1. Strict consistency is the strongest consistency model.
2. In strict consistency, any read operation always returns the value of the most recent write operation.
3. All processes see updates immediately after they occur.
4. This model assumes the existence of a global clock.
5. Strict consistency is difficult to achieve in real distributed systems.
6. An example is reading a file immediately after updating it and always getting the latest value

2. Sequential Consistency

1. Sequential consistency means all processes see operations in the same order.
2. The result of execution is the same as if all operations were executed one after another in some sequence.
3. Different processes may not know the exact real-time order, but they see the same sequence of events.
4. Sequential consistency is weaker than strict consistency.
5. An example is multiple users editing a shared document and seeing updates in the same order.

3. Causal Consistency

1. Causal consistency ensures that causally related operations are seen in the same order by all processes.
2. If one write operation depends on another write, all processes must see them in that order.
3. Independent operations may be seen in different orders by different processes.
4. This model is weaker than sequential consistency.
5. An example is replying to a social media post after reading the original post.

4. FIFO Consistency

1. FIFO consistency is also called PRAM consistency.

2. FIFO consistency ensures that writes from a single process are seen by all processes in the same order.
3. Writes from different processes may be seen in different orders.
4. FIFO consistency is weaker than causal consistency.
5. An example is messages sent by one user appearing in the same order to all receivers.

5. Weak Consistency

1. Weak consistency allows temporary inconsistency in the system.
2. Updates are guaranteed to become visible only after synchronization operations occur.
3. The system does not ensure immediate visibility of writes.
4. Weak consistency improves system performance.
5. An example is a distributed cache where updates are synchronized after some interval.

6. Release Consistency

1. Release consistency is an extension of weak consistency.
2. In this model, synchronization is done using acquire and release operations.
3. Before accessing shared data, a process performs an acquire operation.
4. After updating data, the process performs a release operation.
5. Changes become visible to other processes only after the release operation.
6. An example is lock-based synchronization in shared memory systems.

7. Entry Consistency

1. Entry consistency is a more refined version of release consistency.
2. Each shared variable is associated with a synchronization object.
3. A process must acquire the related synchronization object before accessing the variable.
4. Only the required shared variables are updated during synchronization.
5. Entry consistency reduces unnecessary communication.
6. An example is accessing a database table only after locking it.

Q. Different Client-Centric Consistency Models and difference between them

=>

1. Client-centric consistency models are used in distributed systems where updates may not be immediately visible everywhere.
2. These models ensure that a single client sees data in a consistent way while moving between replicas.
3. Client-centric consistency is useful in systems with weak consistency and replicated data.
4. These models focus on the view of one client rather than all clients in the system.

Client Consistency Model	Meaning	Main Property	Example
Monotonic Read Consistency	If a client reads a value, the client will never see an older value in future reads.	Read values move only forward and never backward.	If a user reads version 5 of a file, later reads will not show version 3.
Monotonic Write Consistency	Write operations from the same client are completed in the order in which they were issued.	Earlier writes are always completed before later writes.	If a user updates profile name first and address later, all servers will process the name update before the address update.
Read Your Writes Consistency	A client always sees its own latest updates in future reads.	A user can immediately see their own changes.	If a user changes a password, the next read will show the new password and not the old one.
Writes Follow Reads Consistency	A write operation by a client is always performed on the latest version that the client has read.	New writes are based on the latest read value.	If a user reads the latest post and then comments on it, the comment will be attached to the latest version of that post.

1. Monotonic Read Consistency

1. Monotonic read consistency ensures that once a client reads a value, future reads will never return an older value.
2. The client always moves forward to newer versions of data.
3. This prevents confusion caused by reading stale data.
4. An example is reading the latest email inbox and never seeing older inbox contents again.

2. Monotonic Write Consistency

1. Monotonic write consistency ensures that write operations from the same client are performed in the correct order.

2. Earlier writes must be completed before later writes.
3. This prevents incorrect ordering of updates.
4. An example is saving a draft before publishing a post.

3. Read Your Writes Consistency

1. Read your writes consistency ensures that after a client performs a write, future reads by the same client will show that updated value.
2. The client does not see old data after making changes.
3. This model improves user experience because users can immediately see their own updates.
4. An example is updating a profile picture and immediately seeing the new picture.

4. Writes Follow Reads Consistency

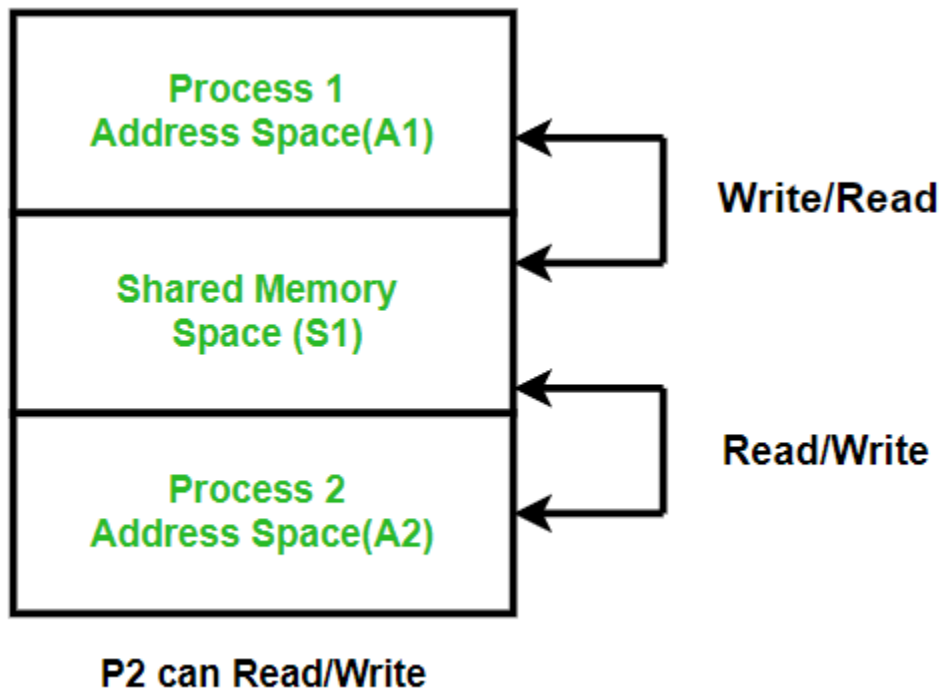
1. Writes follow reads consistency ensures that a client's write is based on the latest version of the data it has read.
2. The system guarantees that the write is applied to the correct and most recent value.
3. This prevents updates from being made on outdated information.
4. An example is reading the latest version of a document and then editing it.

Difference Between Client-Centric Consistency Models

Feature	Monotonic Read	Monotonic Write	Read Your Writes	Writes Follow Reads
Focus	Future reads should not go backward	Writes should remain in order	Client should see its own writes	Writes should be based on latest reads
Type of Operation	Read	Write	Read after Write	Write after Read
Main Benefit	Prevents stale reads	Prevents out-of-order writes	Shows immediate updates to user	Prevents updates on old data
Example	Reading latest email version	Updating profile details in order	Seeing updated password immediately	Editing latest document version

Q. Architecture of shared memory

=>



1. Shared memory architecture is a communication method in distributed systems where multiple processes use a common memory area to exchange data.
2. In this architecture, each process has its own private address space and also has access to a shared memory region.
3. The shared memory region is used for communication between processes.
4. Processes can read data from the shared memory and write data into it.
5. Shared memory provides faster communication because processes do not need to send messages over a network.
6. The operating system creates and manages the shared memory segment.
7. Each process maps the shared memory segment into its own address space.

Components of Shared Memory Architecture

1. Process Address Space

1. Each process has its own private memory area called address space.
2. The private address space contains the process code, variables, stack, and heap.
3. One process cannot directly access the private address space of another process.
4. In the given figure, Process 1 has address space A1 and Process 2 has address space A2.

2. Shared Memory Space

1. Shared memory space is a common memory area accessible to multiple processes.
2. Both Process 1 and Process 2 can read from and write to the shared memory segment S1.

3. Shared memory acts as a bridge for communication between processes.
4. Data written by one process can be read immediately by another process.

Working of Shared Memory Architecture

1. The operating system creates a shared memory region.
2. The shared memory region is attached to the address spaces of the processes.
3. Process 1 writes data into the shared memory area.
4. Process 2 reads the data from the same shared memory area.
5. Similarly, Process 2 can also write data and Process 1 can read it.
6. Both processes can communicate without copying data repeatedly.